

System Design

Interview Cracker question bank

Generated from the Markdown sources in the Interview Cracker repository. Every code snippet in these pages was executed and checked before publication.

Cache Stampede

[Hard] Q. What is a cache stampede, and how do you stop one hot key overwhelming the database?

Answer. A cache stampede happens when a popular entry expires and many requests miss together, so they all fetch or recompute the same value and send a burst to the backing store.

The protections solve different parts of the problem:

- **Request coalescing / single-flight:** one caller refreshes; others wait.
- **Jittered TTLs:** spread expirations so many keys do not fail at once.
- **Stale-while-revalidate:** serve bounded stale data during one refresh.
- **Early refresh:** probabilistically refresh hot data before expiry.

A distributed lock can coordinate hosts, but it needs a short lease and safe ownership; otherwise it becomes another outage source. Load shedding and database limits remain the final backstop because no cache strategy can make the backing service survive arbitrary demand.

System Design Fundamentals

System design interviews have no single right answer. They test whether you can reason about trade-offs out loud, ask about requirements before designing, and justify choices with numbers.

How to run the interview

- 1 **Clarify requirements** (5 min). Functional: what must it do? Non-functional:
 - 2 **Estimate scale** (5 min). Get to a rough QPS and storage figure - it decides
 - 3 **Sketch the high-level design** (10 min). Boxes and arrows: clients, load
 - 4 **Design the data model and API** (10 min).
 - 5 **Go deep** (10 min) on whatever the interviewer pushes on.
 - 6 **Identify bottlenecks** (5 min) and address them.
-

[Easy] Q. How do you do back-of-the-envelope estimation?

Answer. Learn a handful of numbers and round aggressively.

Quantity	Approximate
Seconds in a day	~86,400, call it 10^5
1 million writes/day	~12 writes/second
Memory read	~100 ns
SSD random read	~100 μ s (1000x memory)
Disk seek (HDD)	~10 ms (100x SSD)
Same-datacentre round trip	~0.5 ms
Cross-continent round trip	~150 ms
One character (ASCII)	1 byte

Example - 100 M daily active users each posting twice:

- Writes: 200 M/day / 10^5 s ~ **2,000 writes/second**.
- Peak is typically 2-3x average: **~5,000/s**.
- At 1 KB per post: 200 GB/day, **~73 TB/year**.

That tells you immediately: one database will not hold a year of this, you need sharding, and reads will dominate so you need caching.

[Easy] Q. Vertical vs horizontal scaling?

Answer. Vertical - a bigger machine. Simple, no code changes, but there is a hard ceiling, it is expensive at the top end, and it leaves a single point of failure.

Horizontal - more machines. Effectively unbounded and fault-tolerant, but it forces you to handle statelessness, load balancing, data partitioning and distributed consistency.

The practical answer: **scale vertically until it hurts, then horizontally**. Horizontal scaling adds real complexity, and a surprising number of systems never need it. But design stateless services from the start so

the option stays open.

[Medium] Q. Where do you put caches, and how do they get stale?

Answer. Caching is the highest-leverage optimisation in most designs, because read traffic usually dominates by 10:1 or more.

Layers: browser -> CDN -> load balancer -> application (in-process) -> distributed cache (Redis/Memcached) -> database buffer pool.

Strategies:

- **Cache-aside** (most common) - app checks the cache, misses, reads the
- **Write-through** - write to cache and database together. Consistent, slower
- **Write-behind** - write to cache, flush to the database asynchronously. Fast,

Eviction: LRU is the default and usually right. (A worked LRU cache is in `examples/cpp/data-structures/lru_cache.cpp`.)

Invalidation is the hard part. TTLs are simple but serve stale data for their duration; explicit invalidation on write is precise but easy to miss a path. Watch for the **thundering herd** - a popular key expiring makes thousands of requests hit the database simultaneously. Mitigate with a lock so only one request refills, or by staggering TTLs.

[Medium] Q. Explain replication and sharding.

Answer. They solve different problems and are usually used together.

Replication - copies of the same data. Solves read scaling and availability. Primary-replica is the usual shape: writes go to the primary and propagate to replicas that serve reads. The catch is **replication lag** - a read right after a write may hit a replica that has not caught up, so the user does not see their own change. Fixes: read-your-own-writes routing to the primary, or a version token.

Sharding (partitioning) - splitting different data across machines. Solves write scaling and dataset size.

Strategy	How	Risk
Range	Keys A-M here, N-Z there	Hot spots from uneven distribution
Hash	<code>hash(key) % N</code>	Resharding moves almost everything
Consistent hashing	Keys and nodes on a ring	Adding a node moves only ~1/N of keys
Directory	A lookup service maps key -> shard	The directory is a bottleneck/SPOF

Sharding costs you cross-shard joins and transactions. Choose the shard key so the common query hits one shard.

[Hard] Q. What is CAP, and what does it actually mean in practice?

Answer. In a distributed system you can have at most two of **C**onsistency, **A**vailability and **P**artition tolerance.

The nuance most candidates miss: **partitions are not optional**. Networks fail, so P is a given. The real choice is what you do during a partition:

- **CP** - refuse requests rather than serve possibly-stale data. Banking,
- **AP** - keep serving, reconcile later. Social feeds, DNS, shopping carts -

PACELC extends it usefully: if Partitioned, choose A or C; Else, choose Latency or Consistency. That second half describes normal operation, which is where systems actually spend their time.

Eventual consistency means replicas converge if writes stop. That is weaker than it sounds, which is why stronger models exist - read-your-writes, monotonic reads, causal consistency - and naming the specific guarantee you need is a much better answer than "eventually consistent".

[Medium] Q. When do you add a message queue?

Answer. To decouple a producer from a consumer in time.

It buys you:

- **Asynchrony** - return to the user immediately, do the slow work later. (An
- **Buffering** - a traffic spike queues up instead of overwhelming the consumer.
- **Retries** - a failed message is redelivered rather than lost.
- **Fan-out** - one event, many independent consumers.

The costs are real: eventual consistency (the work is not done when you reply), message ordering is hard across partitions, and you must handle **duplicate delivery** - most queues guarantee at-least-once, not exactly-once. That makes **idempotent consumers** a requirement, not a nicety.

[Medium] Q. Rate limiting - what algorithms would you use?

Answer.

- **Fixed window** - count per calendar minute. Trivial, but allows a 2x burst
- **Sliding window log** - timestamps of every request. Exact, memory-hungry.
- **Sliding window counter** - weighted blend of the current and previous window.
- **Token bucket** - tokens refill at a steady rate; each request spends one.
- **Leaky bucket** - outflow is perfectly constant. Smooths traffic, no bursts.

In a distributed system the limiter state must be shared (typically Redis), which adds a network hop to every request - so a common design is a local limiter with a periodically synchronised global budget.

[Medium] Q. Your service must call a payment provider over the network and retry

on failure. How do you make sure a retry does not charge the customer twice?

Answer. You make the operation **idempotent** so that sending it once or five times has the same effect as sending it once. In practice that means giving the logical operation a stable **idempotency key** that you generate once (before the first attempt) and send with every attempt; the provider stores the result of the first key it has seen and returns that stored result for every repeat, instead of performing the charge again.

The reasoning, and the traps:

- **The key identifies the operation, not the attempt.** You generate it
- **Retries happen because you cannot tell "failed" from "slow".** The
- **Idempotency is a property of the receiver, triggered by the sender.** The
- **Scope the key to the thing you want to repeat.** "Charge order 1234" gets

The trade-off worth stating: idempotency keys make **at-least-once delivery safe**, which is almost always what you want over "at-most-once" (drop on failure). The cost is that the receiver keeps a deduplication record with a retention window, and the key space must be chosen so that legitimate repeat-able operations collide intentionally while distinct operations never do. This is also why HTTP distinguishes `POST` (not idempotent by default) from `PUT/DELETE` (idempotent) - a client can safely retry the latter.

[Medium] Q. Your load balancer fronts 10 replicas. How do you find out when one

stops working - and what do you do about it?

Answer. Health checks, in two flavours: **active** (the LB itself probes each replica - an HTTP GET on a `/healthz` endpoint, or a TCP connect - on a schedule, and removes replicas that fail N times in a row) and **passive** (the LB watches real traffic: if requests routed to a replica start failing or timing out, it stops routing to it). Production systems use both: active checks catch a replica that is up but broken even before traffic would have hit it, and passive checks react instantly to failures that active checks' poll interval cannot see.

The parts an interviewer is looking for next:

- **The probe must be a liveness probe, not a dependency probe.** If
- **Hysteresis in both directions.** Removal should require a few consecutive
- **Ejection beats polling-only.** Passive ejection is the fastest signal you
- **The trap: removing a sick replica shifts its load to the healthy ones.**

Follow-up. "Replicas are flapping: healthy, unhealthy, healthy, every minute. What do you do?" Expected: lengthen the failure streak and/or add hysteresis delay, look for a resource cliff (memory pressure, connection pool exhaustion, a dependency at its limit), and if the cause is external, prefer keeping the replica in and shedding load (circuit breaker) over churning the pool.

Worked Example: Design an API Rate Limiter

A complete worked answer for "design the rate limiter in front of a public API," start to finish. The goal is to show the shape of a good answer: clarify, estimate, pick a data structure on purpose, design the data model, walk the request path, and name the bottlenecks you would actually hit.

1. Clarify requirements

- **What is being limited?** Requests from a client identity (API key or
- **What is the limit?** "N requests per rolling window of W seconds, per
- **What happens at the limit?** Return `429 Too Many Requests` with a
- **Where does it run?** In front of the API servers, on the hot path of
- **Multi-tenant fairness?** A single misbehaving key must not starve others,

The honest scoping line: "I'll design a **fixed-window** counter first because it is simple and good enough, then show the upgrade to **sliding window** and **token bucket** and when each is worth it."

2. Estimate the scale

- 10 M active API keys, 100 M requests/minute peak ~ **1.7 M req/s**.
- Each decision must take < ~2 ms and touch at most one shard.
- State per key is tiny (a counter, or a few counters). 10 M keys x a few

The key insight: this is a **read-mostly, per-key, low-latency** workload. The data is small and hot. That argues strongly for an **in-memory, sharded** design rather than a database on the hot path.

3. The core data structure

Three algorithms, in increasing order of fairness and cost:

- **Fixed window.** One counter per key, reset every window. `count < limit` ?
- **Sliding window log.** Store every request timestamp per key; admit if
- **Sliding window counter (the pragmatic middle).** Keep the current and
- **Token bucket** (the "smooth rate + allow bursts" variant). Each key has a

Decision to state out loud: fixed-window counter for v1 (simple, sharded, $O(1)$); token bucket if the spec allows bursts; sliding-window counter if the boundary burst is a real problem and we want to stay $O(1)$.

4. Data model

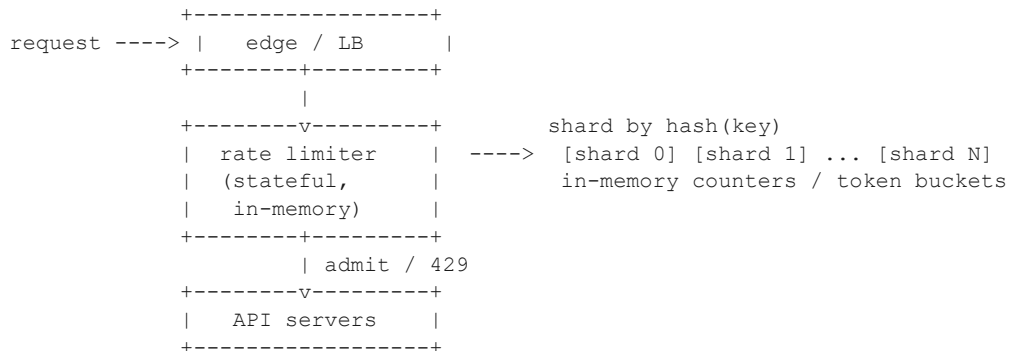
Per key, store the minimum state the chosen algorithm needs:

```
limiter_state (in memory, sharded by key)
  key          STRING    (API key / user id)  -- partition key
  window_start  TIMESTAMP
  count         INT8/INT32
  -- token bucket variant instead:
```

```
tokens          FLOAT      (current bucket level)
last_refill     TIMESTAMP
```

State is small and hot -> **keep it in memory** on the limiter nodes, sharded by hash(key). Persistence is a reliability concern, not a correctness one: if a limiter node dies we can lose that shard's counters and briefly under-count - acceptable for a limiter (failing open for a few seconds beats failing closed and rejecting everyone). Optionally replicate the hot shard to a backup node so a failure is a sub-second failover rather than a reset.

5. The architecture



- The limiter is a thin, **stateful** layer. Every request hashes its key to a
- Because state is per-shard and the decision is local, there is **no**
- 429 responses carry `Retry-After` so well-behaved clients back off.

6. The request path in detail

- 1 Edge extracts the identity (API key / auth token) and hashes it -> shard.
- 2 That shard computes the decision:
- 3 fixed window: if `now` crossed into a new window, `reset count`; then
- 4 token bucket: `refill tokens += R * (now - last_refill)` (capped at B),
- 5 Admit -> forward to the API servers. Reject -> return `429 + Retry-After`

The invariant to name: **the check-and-increment must be atomic per key**. Within one shard that is a single-threaded or lock-protected counter - no race. Across shards a key always maps to exactly one shard, so there is no cross-shard race. This is why "shard by key" is load-bearing, not just a scaling trick.

7. Bottlenecks and what you would do next

- **Hot keys**. A single abusive key hammers one shard. Mitigations: detect
- **Limiter availability**. It is now on the critical path. Run it as a
- **Clock skew**. Windows and token refill depend on `now`. Use a shared,
- **Burst at window boundary** (fixed window) -> the sliding-window-counter or
- **Global limits** ("the whole API gets 5 M req/s") need a second,

8. What the interviewer is grading

- Did you **clarify** limit semantics (per-key, rolling vs fixed, reject vs
- Did you pick the data structure **on purpose** and name the trade-off
- Did you get **sharding by key** and the **atomic per-key counter** right -
- Did you treat the limiter as a **critical component** (availability, fail
- Did you estimate scale and use it to argue for **in-memory** state?

Worked Example: Design a URL Shortener

A complete answer, start to finish, in the order you should actually deliver it. The point is the shape of the answer - the same structure works for any design question.

1. Clarify requirements

Ask before designing. Suggested questions and the answers assumed here:

Functional

- Shorten a long URL into a short one, and redirect from short to long. [yes]
- Custom aliases? Yes, optional.
- Expiry? Yes, optional TTL.
- Analytics on clicks? Basic counts, not real-time.
- User accounts, editing, deletion? Out of scope for now.

Non-functional

- Scale? 100 M new URLs per day.
- Read/write ratio? 100:1 - redirects vastly outnumber creations.
- Latency? Redirect under 100 ms at p99.
- Availability? *Highly available. A redirect failing is worse than a creation
- Consistency? A newly created link should work immediately; analytics may lag.

That last pair is the important trade: **AP for reads, with read-your-writes for the creator.**

2. Estimate the scale

```
Writes: 100 M/day / 105 s ~ 1,000 URLs/second
Reads: 100 x writes ~ 100,000 redirects/second
Peak: ~2x average ~ 200,000 redirects/second

Storage per record:
short code 7 bytes
long URL ~500 bytes
metadata ~100 bytes (created_at, expiry, owner, counter)
-----
~600 bytes, round to 1 KB

100 M/day x 1 KB = 100 GB/day ~ 36 TB/year
```

What these numbers tell us immediately:

- 200 K reads/second cannot come from a database -> **cache is mandatory.**
 - 36 TB/year does not fit one node -> **sharding required.**
 - Reads dominate 100:1 -> optimise the read path above everything else.
-

3. How long must the short code be?

With 62 characters (a-z A-Z 0-9):

Length	Combinations
5	$62^5 \sim 916 \text{ M}$
6	$62^6 \sim 56 \text{ B}$
7	$62^7 \sim 3.5 \text{ trillion}$

At 100 M/day we use 36.5 B/year. Six characters is exhausted in about 18 months; **seven characters gives roughly 95 years of headroom**. Choose 7.

4. API

```
POST /api/v1/urls
{ "long_url": "https://...", "custom_alias": "sale", "ttl_days": 30 }
-> 201 { "short_url": "https://sho.rt/aB3dE7x", "expires_at": "..."}

GET /{code}
-> 301 or 302 to the long URL
-> 404 if unknown or expired
```

301 vs 302 is a real decision. 301 (permanent) is cached by the browser, so subsequent visits never reach your servers - cheaper and faster, but you lose click analytics and cannot ever change the target. **302** keeps every click flowing through you. Given the analytics requirement: **302**.

5. Generating the code

Three options, and the trade-off is the answer:

(a) Hash the URL - `base62(md5(url))[0:7]`. Deterministic, so the same URL gives the same code. But collisions must be detected and resolved by rehashing with a salt, which costs a read before every write.

(b) Random generation - pick 7 random characters, retry on collision. At 3.5 trillion possibilities and billions of records, collisions are rare, but you still need a uniqueness check.

(c) Counter + base62 encoding [yes] - maintain a distributed counter and encode it. **Collision-free by construction**, no read before write, and short codes. The downsides: codes are sequential and therefore guessable and enumerable, and the counter is a coordination point.

Chosen: (c), with mitigations. Use a ticket service (or Redis `INCR`, or ZooKeeper) handing out **ranges** of 1,000 IDs to each application server, so servers only coordinate once per thousand URLs rather than per request. Then XOR/permute the counter before encoding so codes are not obviously sequential.

6. Data model

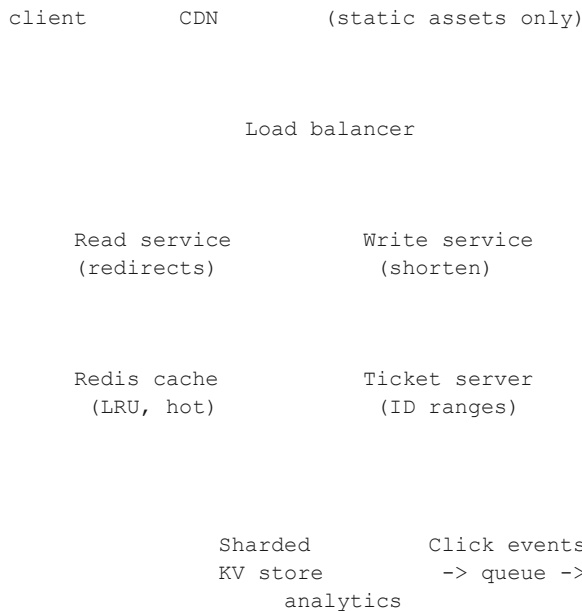
A key-value workload: look up one row by primary key, no joins, no complex queries. That points at a NoSQL store (DynamoDB, Cassandra) rather than a relational one, though a sharded relational database is entirely defensible.

```
urls
code          VARCHAR(7)  PRIMARY KEY / partition key
long_url      TEXT
created_at    TIMESTAMP
expires_at    TIMESTAMP   NULL
```

owner_id VARCHAR NULL

Shard by code using consistent hashing. Every read is by code, so every read hits exactly one shard - no scatter-gather.

7. The architecture



Read and write paths are separate services because their load differs by 100x and they should scale - and fail - independently.

8. The read path in detail

- 1 Request arrives at the read service.
- 2 Look up `code` in Redis. **Expect ~90% hit rate**: link popularity follows a
- 3 On a miss, read the shard, populate the cache, return.
- 4 Emit a click event to a queue (fire-and-forget) - **never** let analytics

Cache sizing: if 20% of URLs are hot, that is 20 M x 1 KB = **20 GB**, which fits comfortably in a small Redis cluster.

9. Bottlenecks and what you would do next

Risk	Mitigation
Cache stampede on a viral link	Single-flight refill; never expire hot keys mid-spike
Ticket server is a SPOF	Replicate it; servers hold a local range so a brief outage is invisible
Hot shard	Consistent hashing with virtual nodes
Abuse (malware, phishing)	Reputation check on creation, blocklist on redirect
Expired-record cleanup	TTL in the store, or a background sweeper

10. What the interviewer is grading

Not the diagram. They want to see that you:

- **asked about requirements before designing** - the most common failure;
- **used numbers** to justify the cache, the shard count and the code length;
- **named trade-offs explicitly** (301 vs 302, counter vs hash) rather than
- **separated read and write paths** once you knew the ratio;
- **knew what you had not solved** and could say so.

Saying "I'd choose X because of Y, though Z would be better if the requirement were W" is worth more than any particular architecture.